

Week 7: Pipelining and Pipeline Hazards

From Single-Cycle MIPS to Overlapped Instruction Execution

September 19, 2026

Where Are We in the Course?

Already discussed

- Digital logic and sequential circuits
- FSMs and control
- ISA concepts and LC-3
- LC-3 multi-cycle microarchitecture
- MIPS single-cycle datapath
- Clock period, CPI and execution time

This week

- Why pipelining?
- Five-stage MIPS pipeline
- Pipeline registers
- Pipeline performance
- Structural hazards
- Data hazards and forwarding
- Control hazards and branch prediction

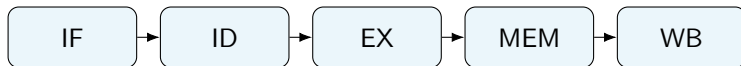
Single-cycle, multi-cycle, and pipelined designs are different microarchitectures for implementing an ISA.

Recall: The Limitation of Single-Cycle MIPS

In a single-cycle processor:

$$T_{\text{program}} = N \times CPI \times T_{\text{clk}}, \quad CPI = 1$$

But the clock period must accommodate the **slowest instruction**.



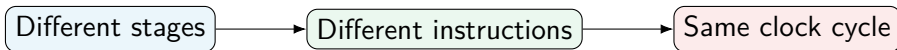
Question: while one instruction is using the ALU, are the instruction memory and register file necessarily busy?

The Key Observation

An instruction uses different hardware resources at different times.

Stage	IF	ID	EX	MEM	WB
Main work	Instr. memory	Reg. read	ALU	Data memory	Reg. write

If instruction I_1 has moved to EX, the IF hardware can potentially fetch I_3 .



Pipelining improves resource utilization by overlapping instruction execution.

Assembly-Line Analogy

Suppose making one product requires five steps:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$$

Without pipelining

Finish one product before starting the next.

$$P_1 : A B C D E$$

$$P_2 : \quad A B C D E$$

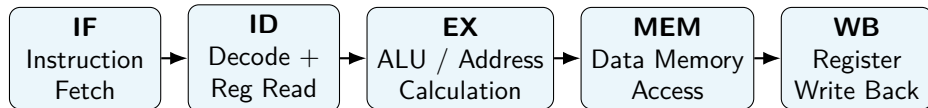
With pipelining

Different products occupy different stages simultaneously.

	1	2	3	4	5
P_1	A	B	C	D	E
P_2		A	B	C	D
P_3			A	B	C

Important: pipelining overlaps work; it does not make one instruction's logic magically disappear.

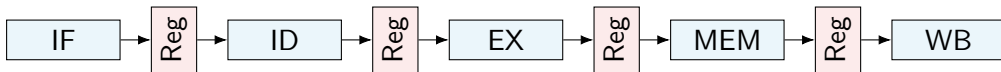
Five Stages of the MIPS Pipeline



- IF: fetch instruction and form next sequential PC.
- ID: decode and read source registers.
- EX: perform ALU operation / effective-address calculation / branch comparison.
- MEM: access data memory when required.
- WB: write result into the register file.

From Single-Cycle Datapath to Pipeline

Why can we not simply overlap instructions in the existing combinational datapath?



Pipeline registers preserve each instruction's intermediate values at stage boundaries:



At every clock edge, each stage hands its result to the next stage.

Why Pipeline Registers Are Necessary

Recall our earlier sequential-circuit idea:

Register $\rightarrow$ Combinational Logic $\rightarrow$ Register

Without stage registers, values from several instructions would mix while propagating through the datapath.

- They separate one pipeline stage from the next.
- They hold data stable for the next clock cycle.
- They allow different instructions to occupy different stages simultaneously.
- They also carry the **control signals** needed later.

Pipeline registers are programmer-invisible microarchitectural state.

What Must Each Pipeline Register Hold?

Register	Typical information carried
IF/ID	Instruction, next-PC information
ID/EX	Register operands, sign-extended immediate, register numbers, control bits
EX/MEM	ALU result, store data, destination register, branch-related information, control bits
MEM/WB	Memory data or ALU result, destination register, write-back control

It is not enough to pipeline only data. The instruction's control information must reach the stage where it is used.

Pipeline Timing: First Example

Consider independent instructions:

```
I1: add $t0,$t1,$t2
I2: sub $t3,$t4,$t5
I3: and $t6,$t7,$t8
I4: or  $s0,$s1,$s2
```

	1	2	3	4	5	6	7	8
I_1	IF	ID	EX	MEM	WB			
I_2		IF	ID	EX	MEM	WB		
I_3			IF	ID	EX	MEM	WB	
I_4				IF	ID	EX	MEM	WB

After filling the pipeline, ideally **one instruction completes every cycle**.

Latency vs. Throughput

Latency

Time from the start of one instruction until that instruction finishes.

For a five-stage pipeline, one instruction still traverses five stages.

Throughput

Rate at which completed instructions emerge.

After pipeline fill:

$$\text{Ideal throughput} \approx \frac{1}{T_{\text{clk}}}$$

Pipelining primarily improves throughput, not the latency of a single instruction.

Ideal Pipeline Execution Time

For a k -stage pipeline executing N instructions, with no stalls:

$$T_{\text{pipe}} = (k + N - 1)T_{\text{clk}}$$

Why?

- First instruction requires k cycles to emerge.
- Each additional instruction ideally completes one cycle later.

For $k = 5$:

$$T_{\text{pipe}} = (N + 4)T_{\text{clk}}$$

For large N :

$$CPI_{\text{ideal}} \rightarrow 1$$

What Sets the Pipeline Clock Period?

If stage delays are

$$T_{IF}, T_{ID}, T_{EX}, T_{MEM}, T_{WB}$$

then approximately

$$T_{\text{clk}} \geq \max(T_{IF}, T_{ID}, T_{EX}, T_{MEM}, T_{WB}) + T_{\text{reg}}$$

where T_{reg} represents pipeline-register overhead.

- The slowest stage limits the clock frequency.
- Unequal stage delays reduce the potential speedup.
- More stages can reduce logic per stage, but increase register overhead.

Why Ideal $CPI = 1$ Is Not Guaranteed

The timing table assumed that every instruction could advance every cycle.
But three things can prevent this:

Structural
resource conflict

Data
dependency

Control
next PC uncertain

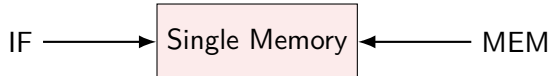
Hazard: a condition that prevents the next instruction from executing in its intended cycle.

Structural Hazard

A structural hazard occurs when two simultaneous pipeline operations need the **same hardware resource**.

Example: one unified single-port memory.

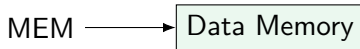
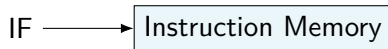
- Instruction I_1 in MEM needs memory for a load/store.
- Instruction I_4 in IF simultaneously needs memory to fetch an instruction.
- Both cannot use the same single port in the same cycle.



Avoiding the Classic MIPS Structural Hazard

A common five-stage MIPS organization provides separate paths/resources for:

- instruction fetch, and
- data-memory access.



Structural hazards are not inevitable; they depend on the chosen hardware organization.

Data Dependency vs. Data Hazard

Consider:

```
I1: add $t0,$t1,$t2  
I2: sub $t3,$t0,$t4
```

I_2 **depends** on the value produced by I_1 .

- Dependency is a property of the program.
- A hazard occurs when the microarchitecture cannot satisfy that dependency at the required time.

Dependencies cannot simply be “removed” by hardware; the processor must preserve the program’s required ordering of values.

Three Names You Will See: RAW, WAR, WAW

	Meaning	Example relation
RAW	Read After Write	Later instruction reads a value produced earlier
WAR	Write After Read	Later instruction writes a location that an earlier instruction must read first
WAW	Write After Write	Two instructions write the same destination; order matters

For the classic **in-order five-stage MIPS pipeline** considered here:

RAW is the relevant register data hazard

Why are WAR and WAW absent here? Reads occur in order in ID and writes occur in order in WB.

RAW Hazard: Where Is the Value?

```
I1: add $t0,$t1,$t2  
I2: sub $t3,$t0,$t4
```

	1	2	3	4	5
I_1	IF	ID	EX	MEM	WB
I_2		IF	ID	EX	MEM

At I_2 's ID stage, the new value of $\$t0$ has not yet reached WB.

I_2 reads the old value in ID because I_1 has not yet written the new value back.

This is a RAW hazard.

First Solution: Stall (Without Forwarding)

Consider:

I_1 : add \$t0,\$t1,\$t2

I_2 : sub \$t3,\$t0,\$t4

Without forwarding, I_2 must wait until I_1 writes \$t0 back to the register file.

	1	2	3	4	5	6	7	8
I_1	IF	ID	EX	MEM	WB			
I_2		IF	ID	ID (stall)	ID (read)	EX	MEM	WB

- I_2 reaches ID in cycle 3 but needs the new value of \$t0.
- Without forwarding, I_2 is held in ID until I_1 reaches WB.
- In cycle 5:

$I_1 : WB$		$I_2 : ID$
------------	--	------------

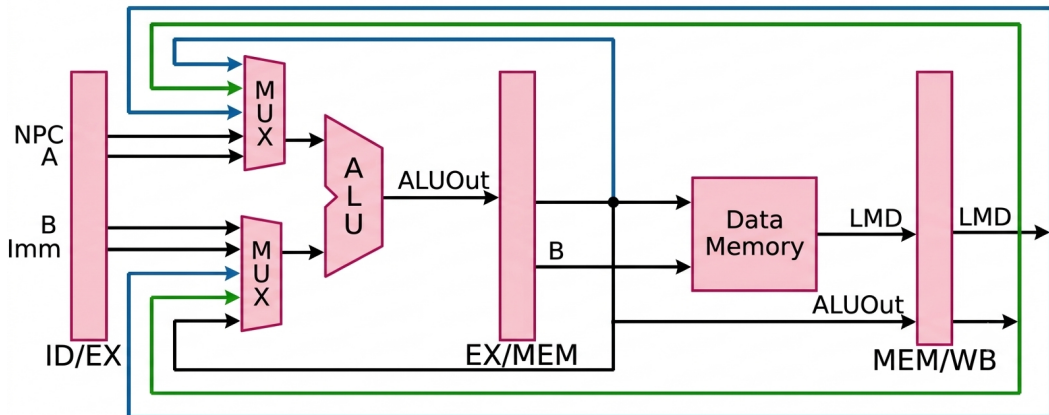
- With the conventional MIPS register-file timing, the value written in WB can be read in ID in the **same clock cycle**.
- Therefore, I_2 enters EX in cycle 6.

Forwarding Paths

For an ALU instruction, the result is actually computed at the end of EX.

Why wait until WB before allowing the next ALU instruction to use it?

Forwarding routes a result directly to a later instruction that needs it.



Example: ALU-to-ALU Forwarding

```
I1: add $t0,$t1,$t2
I2: sub $t3,$t0,$t4
```

	1	2	3	4	5	6
I_1	IF	ID	EX	MEM	WB	
I_2		IF	ID	EX	MEM	WB

At cycle 4:

- I_1 's ALU result is already in EX/MEM.
- I_2 needs that value at its ALU input.
- Forward EX/MEM $\rightarrow$ EX.

For this dependency, forwarding eliminates the stall.

A More Difficult Case: Load-Use Hazard

```
I1: lw    $t0,0($t1)
I2: add   $t2,$t0,$t3
```

When is the loaded value actually available?

	1	2	3	4	5
I_1	IF	ID	EX	MEM	WB
I_2		IF	ID	EX	MEM

I_2 needs the operand during EX in cycle 4, but I_1 obtains the memory data only at the end of MEM in cycle 4.

Forwarding alone is not enough

Load-Use Solution: One Stall + Forwarding

	1	2	3	4	5	6	7
I_1	IF	ID	EX	MEM	WB		
I_2		IF	ID	Stall	EX	MEM	WB

Now I_2 reaches EX one cycle later, when the loaded data can be forwarded.

Classic result: an immediately dependent instruction after lw normally requires one bubble in the basic five-stage pipeline, even with forwarding.

Compiler Instruction Scheduling

Compiler scheduling reorders instructions while preserving program semantics. The moved instruction must be independent of the load and consumer and must not change observable program behavior.

This is **static scheduling**: the compiler chooses the order before the program executes.

Hardware vs. Compiler: Who Handles the Hazard?

Technique	Performed by		Main idea
Forwarding / bypassing	Hardware		Use result before WB
Hazard detection + stall	Hardware		Wait when value is too late
Instruction scheduling	Compiler		Move independent work into delay
Dynamic scheduling	Processor hardware	hard-	Reorder at run time (advanced)

A load-use pair needs a hardware stall if it remains adjacent, but compiler scheduling can sometimes separate the pair and avoid wasting that cycle.

Data Hazard Summary

Situation	Typical action
Independent instructions	No action
ALU result needed by next ALU instruction	Forwarding
Older result needed by a later ALU instruction	Forwarding from appropriate later stage
Load immediately followed by dependent instruction	One stall + forwarding
No forwarding hardware	More stalls may be required

Now Consider a Branch

```
beq $t0,$t1,TARGET  
add $s0,$s1,$s2  
sub $s3,$s4,$s5  
TARGET: ...
```

When fetching the instruction after beq, the processor needs to know:

What should the next PC be?

But the branch condition may not yet have been evaluated.

A control hazard occurs because the identity of the next instruction is uncertain.

Where Is a Branch Resolved?

In a straightforward pipeline, branch processing may require:

- reading source registers,
- comparing operands,
- calculating the branch target,
- selecting the correct next PC.

If the decision is known only after later stages, meanwhile fetched instructions may be from the **wrong path**.

Later branch resolution $\implies$ more wrong-path instructions $\implies$ larger penalty

Stall Until the Branch Is Known

Simplest strategy:

Fetch branch → wait → resolve branch → fetch correct PC

Advantage:

- conceptually simple,
- no wrong-path instructions proceed.

Disadvantage:

- every branch introduces lost cycles.

Branches are frequent enough that repeatedly waiting can significantly increase CPI.

Fetch Sequentially, Then Flush If Wrong

Another strategy is to continue fetching the fall-through path.

If branch is **not taken**: useful work was done.

If branch is **taken**: younger wrong-path instructions must be discarded.

Flush = invalidate instructions fetched from the wrong path

A convenient implementation is to turn those instructions into bubbles.

Can We Resolve the Branch Earlier?

Branch penalty depends strongly on the stage in which the branch is resolved.

Possible improvement:

- move comparison logic earlier,
- calculate the branch target earlier,
- update PC sooner.

But:

- earlier stages become more complex,
- additional forwarding may be required for branch operands,
- the stage delay itself may increase.

Microarchitecture is a trade-off: reducing branch penalty can increase hardware complexity or stage delay.

Instead of Waiting: Predict

A processor can **predict** the branch outcome and continue fetching.

Two broad approaches:

Static

- fixed rule
- e.g., predict not taken
- no execution history required

If prediction is correct, useful work continues. If wrong, wrong-path instructions are flushed.

Dynamic

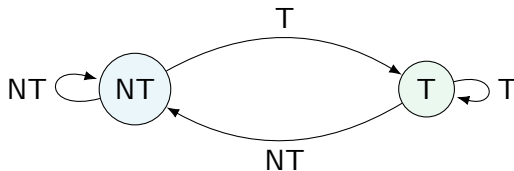
- use previous behavior
- maintain hardware state
- adapt to branch history

One-Bit Branch Predictor

A simple predictor stores one bit for a branch:

0 = predict not taken, 1 = predict taken

After the actual branch outcome is known, update the bit to the latest outcome.



Weakness: one unusual outcome immediately reverses the prediction.

Why One Bit Can Be Poor for Loops

Suppose a loop branch behaves:

$$T, T, T, T, NT$$

At loop exit, a 1-bit predictor changes from T to NT.

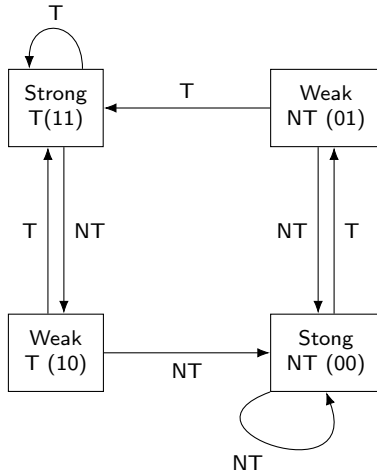
When the loop is encountered again, the first iteration is predicted NT even though it is taken.

Thus a repeated loop can suffer:

- one misprediction near exit, and
- another near the next entry.

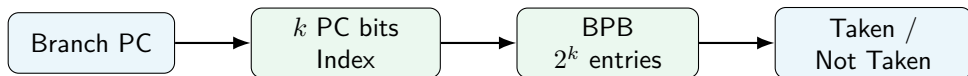
Can we require stronger evidence before reversing the prediction?

Two-Bit Saturating Predictor



Branch Prediction Buffer (BPB)

A Branch Prediction Buffer stores the recent behavior of branch instructions.



- k bits from the branch PC index a BPB containing 2^k **entries**.
- Each entry may contain a **1-bit or 2-bit predictor**.
- Example: $k = 10$ gives

$$2^{10} = 1024 \text{ BPB entries.}$$

- For 32-bit word-aligned MIPS instructions, the lowest two PC bits are normally zero, so a simple implementation could use bits such as

$$PC[11 : 2]$$

as a 10-bit index.

BPB uses the branch PC to predict Taken or Not Taken.

BPB Indexing and Its Limitation

Why use only k PC bits?

- A full 32-bit PC does not require a 2^{32} -entry table.
- Use k PC bits as the index:

$$k \text{ bits} \Rightarrow 2^k \text{ entries}$$

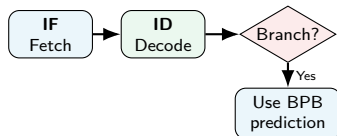
- Examples:

$$k = 5 \Rightarrow 32 \text{ entries}$$

$$k = 10 \Rightarrow 1024 \text{ entries}$$

- Fewer entries increase the chance that different branch PCs map to the same entry.

Limitation of this BPB organization

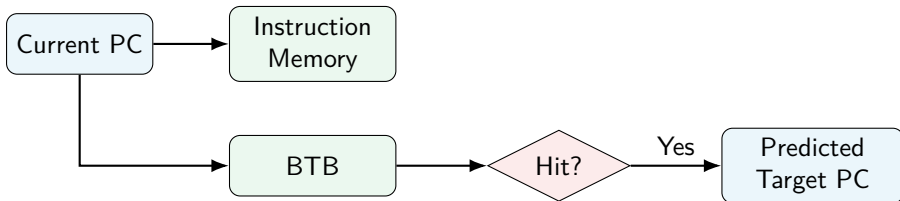


- Branch is identified after decode.
- Meanwhile, the next sequential instruction may already be fetched.
- Thus this organization can still incur a **one-cycle fetch penalty**.

BPB predicts direction, but does not itself provide the target during IF.

Branch Target Buffer (BTB)

A Branch Target Buffer is accessed using the **current PC** during **IF**.



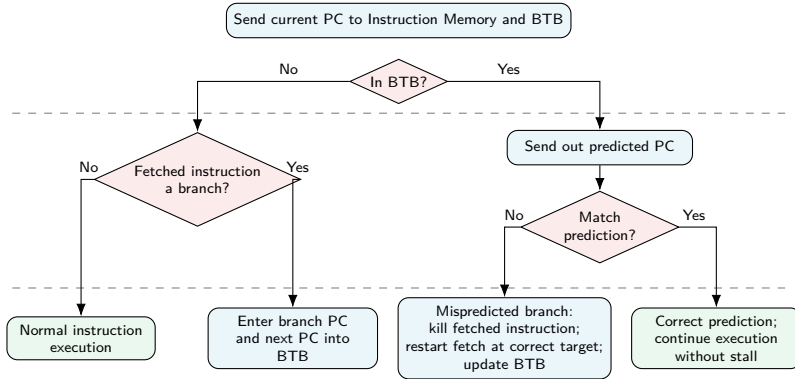
A typical BTB entry contains:

Branch PC/Tag	+	Predicted Target PC	+	Prediction Information
---------------	---	---------------------	---	------------------------

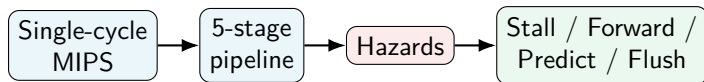
Thus the BTB can indicate that the current PC corresponds to a previously seen branch and provide its predicted target without waiting for ID.

BTB answers during IF: Is this a known branch, and where should fetching continue if predicted taken?

BTB Operation in the Pipeline



Big Picture



Goal: overlap instructions while preserving the exact behavior required by the ISA

- Structural hazard: hardware resource conflict.
- Data hazard: required operand not available at the needed time.
- Control hazard: next instruction address is uncertain.

Supporting textbooks

- Patterson & Hennessy, *Computer Organization and Design*: pipelining and hazards.
- Harris & Harris, *Digital Design and Computer Architecture*: processor microarchitecture and pipelining.